Free Form Testing Agent

Document: 04_action_selection_and_exploration.md

Version: 1.0
Status: Draft
Author: Kalpesh
Date: July 2026

---

1. Purpose

This document defines the architecture and implementation strategy for action selection, deterministic exploration, persistence, failure detection, validation, requirement modeling, and AI-guided testing in the Free Form Testing Agent (FFTA).

The immediate goal is to complete the deterministic exploration loop:

```
Action Selection
        ↓
Single Exploration Step
        ↓
State Observation
        ↓
Transition Recording
        ↓
Failure Detection
        ↓
Memory Update
        ↓
Next Action Selection
        ↓
Repeat
```

The architecture deliberately postpones serious AI integration until the deterministic testing foundation and persistence layer are complete.

The implementation order is:

```
1. Deterministic Action Selection
        ↓
2. Single Exploration Step
        ↓
3. Closed Coordinator Loop
        ↓
4. Persistence and Resume
        ↓
5. Deterministic Failure Detection
        ↓
6. Validation Architecture
        ↓
7. Requirement Model
        ↓
8. AI Strategy Integration
        ↓
9. AI-Guided Testing
```

The central architectural principle is:

> AI should improve testing strategy, not replace deterministic execution infrastructure.

---

2. Why Action Selection Is a Separate Subsystem

The application driver discovers what actions are available.

The action selector decides which action should be executed next.

These are different responsibilities.

The driver answers:

```
What can the agent do?
```

The selector answers:

```
What should the agent do next?
```

This separation is essential because multiple exploration strategies can operate on the same application model.

For example:

```
Available Actions
        |
        +---- BFS Selector
        |
        +---- DFS Selector
        |
        +---- Random Selector
        |
        +---- Heuristic Selector
        |
        +---- AI Selector
```

The application driver does not need to know which strategy is active.

The coordinator does not need to understand how a strategy chooses actions.

The action selector becomes a replaceable decision component.

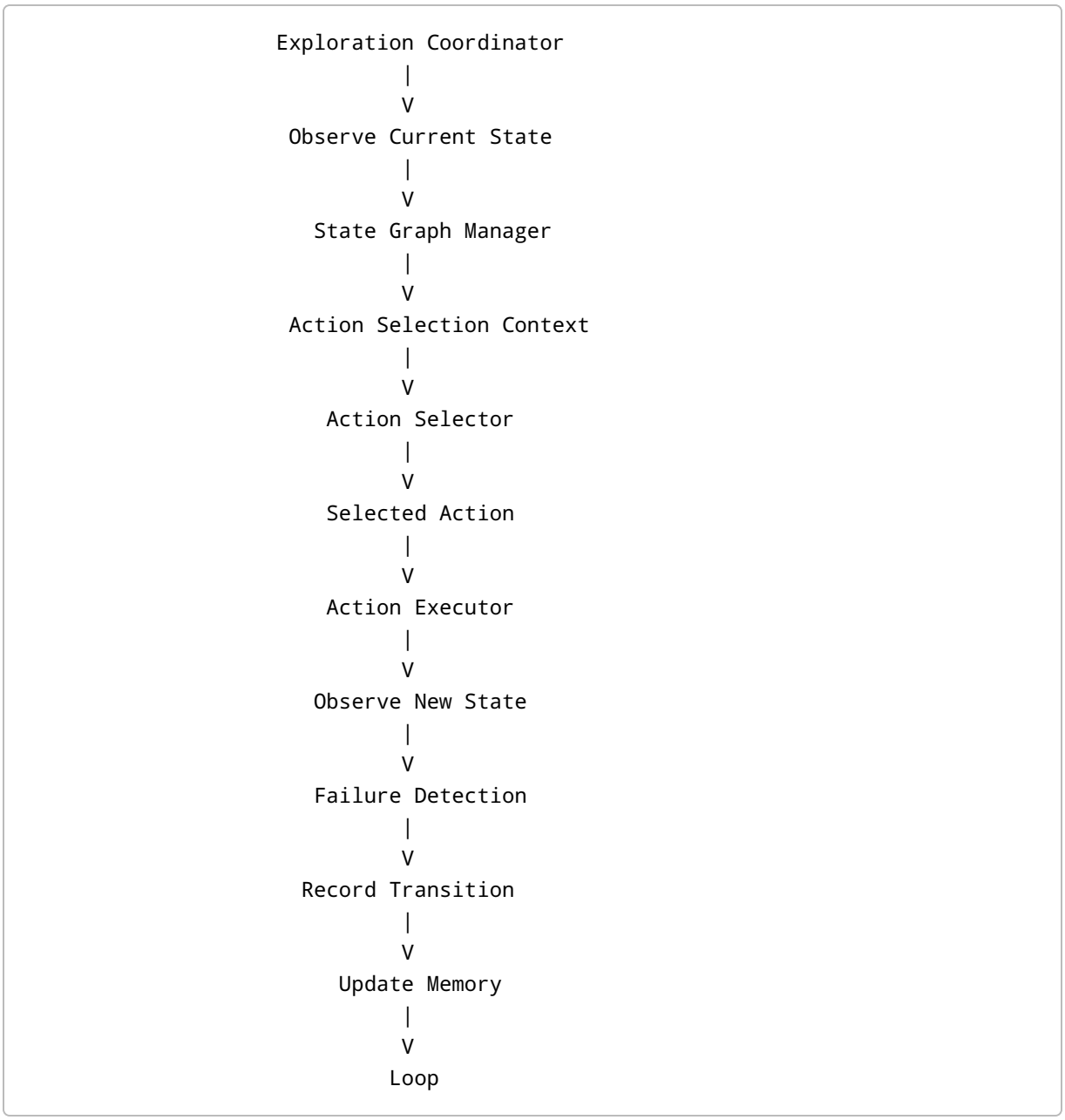
---

3. Architectural Position

The action selector operates between the state graph and the execution engine.

```
Application Driver
        ↓
Observe Current State
        ↓
State Graph
        ↓
Available Actions
        ↓
Action Selector
        ↓
Selected Action
        ↓
Action Executor
        ↓
Observe Result
```

The complete deterministic exploration architecture is:



---

4. Core Action Model

An action represents one operation that the agent can execute against the application.

The action model must be independent of the automation technology.

Example:

```python
@dataclass(frozen=True)
class Action:
    action_type: ActionType
    target_id: str
    value: str | None = None
```

Possible action types:

```python
class ActionType(Enum):
    CLICK = "click"
    DOUBLE_CLICK = "double_click"
    TEXT_INPUT = "text_input"
    KEY_PRESS = "key_press"
    HOTKEY = "hotkey"
    SCROLL = "scroll"
    WAIT = "wait"
```

For the initial Calculator implementation, the primary action is:

```
CLICK
```

Example:

```
Action(
    action_type=ActionType.CLICK,
    target_id="num7Button"
)
```

---

5. Action Identity

Every executable action must have a deterministic identity.

Example:

```
CLICK:num7Button
CLICK:plusButton
CLICK:equalButton
```

An action identity should be derived from stable properties.

Recommended format:

```
<ActionType>:<TargetId>:<Value>
```

Examples:

```
CLICK:num7Button
CLICK:plusButton
TEXT_INPUT:searchBox:hello
KEY_PRESS:mainWindow:ENTER
```

The identity is used for:

- Action comparison
- Coverage tracking
- Transition uniqueness
- Persistence
- Replay
- Failure reproduction
- Exploration scheduling

---

6. Action Candidate

An available action and a selected action are not necessarily the same concept.

The driver discovers raw actions.

The exploration system converts them into action candidates.

Example:

```python
@dataclass
class ActionCandidate:
    action: Action
    source_state_id: str
    discovered_at: datetime
    execution_count: int = 0
    failure_count: int = 0
    score: float = 0.0
```

This model allows future selectors to reason about:

- Whether an action was already executed
- How many times it was executed
- Whether it previously failed
- Whether it produced a new state
- Whether it produced a bug
- AI-generated priority

---

7. Action Selection Context

Selectors should not directly access every internal system component.

Instead, they receive a structured selection context.

```
@dataclass
class ActionSelectionContext:
    current_state: ApplicationState
    available_actions: list[Action]
    state_graph: StateGraph
    exploration_memory: ExplorationMemory
    session: ExplorationSession
```

Future versions may include:

```
@dataclass
class ActionSelectionContext:
    current_state: ApplicationState
    available_actions: list[Action]
    state_graph: StateGraph
    exploration_memory: ExplorationMemory
    session: ExplorationSession
    requirements: list[Requirement]
    known_failures: list[Failure]
    historical_sessions: list[SessionSummary]
```

The selector must treat the context as input.

It should not execute actions itself.

---

8. Action Selector Interface

All action selection strategies should implement the same interface.

```
from abc import ABC, abstractmethod


class ActionSelector(ABC):

    @abstractmethod
    def select_action(
        self,
        context: ActionSelectionContext
    ) -> Action | None:
        pass
```

The output is:

```
Action
```

or:

```
None
```

Returning `None` means:

```
No valid action is currently available.
```

The selector must not:

- Click controls
- Modify application state
- Store transitions
- Detect failures
- Launch applications
- Restart applications

Its only responsibility is selection.

---

9. Deterministic Action Selection

The first selector must be completely deterministic.

Given:

```
Same State
+
Same Available Actions
+
Same Exploration Memory
```

the selector must return:

```
Same Selected Action
```

This property is critical for:

- Debugging
- Reproducibility
- Unit testing
- Replay
- Failure analysis

The initial selector should not use:

- Randomness
- LLMs
- Time-dependent scoring
- Unstable UI ordering

---

10. Initial Selection Strategy

The initial deterministic strategy is:

```
1. Get all available actions.
2. Normalize action ordering.
3. Find actions not yet explored from the current state.
4. Select the first unexplored action.
5. Return None if all actions are explored.
```

Pseudo-code:

```
def select_action(context):

    actions = sorted(
        context.available_actions,
        key=lambda action: action.identity
    )

    for action in actions:

        if not context.state_graph.has_explored(
            context.current_state.id,
            action
        ):
            return action

    return None
```

This creates predictable exploration.

---

11. Why Stable Ordering Is Required

UI automation frameworks may return controls in different orders.

Example run one:

```
1
2
3
+
=
```

Example run two:

```
+
1
=
3
2
```

If the selector uses raw discovery order, exploration becomes nondeterministic.

Therefore, actions must be normalized before selection.

Possible ordering keys:

```
Automation ID
Action Type
Control Name
Target ID
```

Recommended initial ordering:

```
sorted(
    actions,
    key=lambda action: (
        action.action_type.value,
        action.target_id,
        action.value or ""
    )
)
```

---

12. State-Action Coverage

Exploration coverage must be tracked per state.

The system should not ask:

```
Was this button ever clicked?
```

Instead, it should ask:

```
Was this action executed from this specific state?
```

For example:

```
State S0:
Display = 0
```

```
Action:
CLICK num7Button
```

is different from:

```
State S5:
Display = 100

Action:
CLICK num7Button
```

Therefore, the exploration unit is:

```
(State, Action)
```

A state-action pair is considered explored when an execution attempt has been recorded.

---

13. Exploration Memory

The exploration memory stores runtime knowledge used by selectors.

Example:

```
class ExplorationMemory:

    explored_state_actions: set[tuple[str, str]]

    visited_states: set[str]

    failed_actions: set[tuple[str, str]]

    current_path: list[Transition]
```

The memory answers questions such as:

```
Has this state been visited?

Has this action been executed from this state?

Did this action previously fail?
```

```
What path produced the current state?

Which states still contain unexplored actions?
```

---

14. Single Exploration Step

Before building a continuous loop, the framework must support exactly one exploration step.

The single-step operation is the smallest complete testing transaction.

It performs:

```
1. Observe current state.
2. Register current state.
3. Discover available actions.
4. Build selection context.
5. Select one action.
6. Execute one action.
7. Observe resulting state.
8. Detect failures.
9. Register resulting state.
10. Record transition.
11. Update memory.
12. Return step result.
```

This operation should be implemented independently of the continuous coordinator loop.

---

15. Exploration Step Result

Every step should return a structured result.

```python
@dataclass
class ExplorationStepResult:
    source_state: ApplicationState
    selected_action: Action | None
    target_state: ApplicationState | None
    transition: Transition | None
```

```
    failures: list[Failure]
    status: ExplorationStepStatus
```

Possible statuses:

```python
class ExplorationStepStatus(Enum):
    COMPLETED = "completed"
    NO_ACTION_AVAILABLE = "no_action_available"
    ACTION_FAILED = "action_failed"
    APPLICATION_CRASHED = "application_crashed"
    APPLICATION_HUNG = "application_hung"
    OBSERVATION_FAILED = "observation_failed"
```

---

16. Single-Step Execution Flow

```
START STEP
    |
    V
Observe Source State
    |
    V
Register State
    |
    V
Discover Actions
    |
    V
Build Selection Context
    |
    V
Select Action
    |
    +------------------+
    |                  |
 Action             No Action
    |                  |
    V                  V
Execute            Return
    |              NO_ACTION
    V
Observe Target State
    |
    V
```

```
Detect Failure
    |
    V
Record Transition
    |
    V
Update Memory
    |
    V
Return Step Result
```

---

17. Exploration Coordinator

The coordinator owns the closed exploration loop.

Its responsibility is orchestration.

The coordinator repeatedly calls:

```
explore_one_step()
```

It does not contain exploration strategy logic.

Example:

```
while session.is_active:

    result = explorer.explore_one_step()

    if should_stop(result):
        break
```

The architecture should be:

```
Coordinator
    |
    V
Single Step Explorer
    |
    +---- State Observer
    |
```

```
    +---- Action Selector
    |
    +---- Action Executor
    |
    +---- Failure Detector
    |
    +---- Graph Manager
    |
    +---- Memory
```

---

18. Closed Coordinator Loop

The complete deterministic loop is:

```
Start Session
      |
      V
Observe Initial State
      |
      V
Select Action
      |
      V
Execute Action
      |
      V
Observe Result
      |
      V
Detect Failure
      |
      V
Record Transition
      |
      V
Update Memory
      |
      V
Check Stop Conditions
      |
      +------ Stop ------> Finish Session
      |
      V
Restore Exploration Target
```

```
      |
      V
Select Next Action
      |
      V
Repeat
```

---

19. The Restoration Problem

A critical challenge in graph exploration is that the agent physically exists in only one application state at a time.

Suppose the graph contains:

```
        S0
       /
      A    B
     /
    S1      S2
```

After exploring:

```
S0 --A--> S1
```

the application is physically in:

```
S1
```

To explore:

```
S0 --B--> S2
```

the agent must return to:

```
S0
```

Therefore, deterministic exploration requires state restoration.

---

20. State Restoration

The initial restoration strategy should use replay.

```
Restart Application
        ↓
Observe Initial State
        ↓
Replay Known Path
        ↓
Reach Target State
        ↓
Verify Target State
        ↓
Continue Exploration
```

Example:

```
Target State: S3

Known Path:

S0
 ↓ CLICK 7
S1
 ↓ CLICK +
S2
 ↓ CLICK 2
S3
```

The coordinator restarts the application and replays the path.

---

21. Replay Verification

Replay must never assume success.

After each replayed action:

```
Expected State
      vs
Actual State
```

must be compared.

If:

```
Expected State Hash != Actual State Hash
```

the system records:

```
ReplayMismatchFailure
```

Replay verification is essential because application behavior may be:

- Nondeterministic
- Time-dependent
- Data-dependent
- Corrupted
- Changed between runs

---

22. Exploration Termination Conditions

The coordinator must stop deterministically.

Possible stop reasons:

```
NO_UNEXPLORED_ACTIONS
MAX_ACTIONS_REACHED
MAX_STATES_REACHED
MAX_DURATION_REACHED
MAX_DEPTH_REACHED
APPLICATION_CRASHED
APPLICATION_UNRECOVERABLE
MANUAL_STOP
INTERNAL_ERROR
```

The final session should always record the stop reason.

---

23. Persistence Before Serious AI Integration

Persistence should be implemented before serious AI integration.

The reason is architectural rather than merely operational.

Without persistence, AI can reason only about:

```
Current Process Memory
```

With persistence, AI can reason about:

```
Current State
+
Current Graph
+
Previous Sessions
+
Historical Failures
+
Known Workflows
+
Coverage Gaps
+
Previous AI Decisions
```

This makes AI substantially more useful.

The development sequence should therefore be:

```
Deterministic Exploration
        ↓
Persistence
        ↓
Failure Detection
        ↓
Validation
        ↓
Requirements
        ↓
AI Integration
```

---

24. Persistence Architecture

The persistence layer must save:

```
State Graph
Exploration Memory
Sessions
States
Transitions
Actions
Failures
Screenshots
Replay Paths
Coverage
```

Recommended architecture:

```
Runtime Components
        |
        V
Repository Interfaces
        |
        V
Persistence Layer
        |
        +---- SQLite
        |
        +---- File Storage
```

---

25. Repository Interfaces

Core components should not directly execute SQL.

Use repository abstractions.

Example:

```
class StateRepository:

    def save(self, state):
        pass
```

```
    def get(self, state_id):
        pass

    def find_by_hash(self, state_hash):
        pass
```

Additional repositories:

```
SessionRepository
StateRepository
TransitionRepository
FailureRepository
ScreenshotRepository
MemoryRepository
```

---

26. Persistence Storage Responsibilities

SQLite

Store structured information:

```
Sessions
States
Actions
Transitions
Failures
Coverage
Requirements
AI Decisions
```

File Storage

Store large artifacts:

```
Screenshots
Logs
Reports
```

```
Graph Exports
Generated Tests
```

Recommended structure:

```
storage/

├── database/
│   └── ffta.db
│
├── sessions/
│   └── <session_id>/
│       ├── screenshots/
│       ├── logs/
│       ├── reports/
│       └── graph/
│
└── exports/
```

---

27. Session Model

Every exploration run must be represented as a session.

```python
@dataclass
class ExplorationSession:
    session_id: str
    application_id: str
    started_at: datetime
    completed_at: datetime | None
    status: SessionStatus
    stop_reason: StopReason | None
```

Possible statuses:

```
CREATED
RUNNING
PAUSED
INTERRUPTED
COMPLETED
FAILED
```

28. Resume Interrupted Exploration

An interrupted session should be resumable.

Resume flow:

```
Load Session
      ↓
Load State Graph
      ↓
Load Exploration Memory
      ↓
Load Pending Exploration Targets
      ↓
Launch Application
      ↓
Restore Required State
      ↓
Verify State
      ↓
Continue Exploration
```

The system must not restart exploration from zero unless explicitly requested.

---

29. Persistence Checkpoints

The framework should save progress frequently.

Recommended checkpoint points:

```
After State Discovery
After Transition Creation
After Failure Detection
After Memory Update
After Exploration Step
Before Shutdown
```

The safest initial approach is:

> Persist after every completed exploration step.

This is slower than batch persistence but provides stronger recovery guarantees during early development.

Optimization can be added later.

---

30. Deterministic Failure Detection

Failure detection must initially be rule-based and deterministic.

The system should detect failures without requiring an LLM.

Initial failure categories:

```
Application Crash
Application Hang
Window Disappearance
Action Execution Failure
Observation Failure
Replay Mismatch
Unexpected Error Dialog
```

---

31. Failure Model

```python
@dataclass
class Failure:
    failure_id: str
    failure_type: FailureType
    session_id: str
    source_state_id: str | None
    action: Action | None
    evidence: dict
    timestamp: datetime
```

Failure types:

```python
class FailureType(Enum):
    CRASH = "crash"
    HANG = "hang"
    WINDOW_DISAPPEARED = "window_disappeared"
    EXECUTION_FAILURE = "execution_failure"
```

```
    OBSERVATION_FAILURE = "observation_failure"
    REPLAY_MISMATCH = "replay_mismatch"
    UNEXPECTED_ERROR_DIALOG = "unexpected_error_dialog"
```

---

32. Crash Detection

A crash occurs when:

```
Application process existed before action
```

and:

```
Application process no longer exists after action
```

Detection:

```
Process Alive Before Action = True
Process Alive After Action = False
```

Result:

```
CRASH
```

Evidence should include:

```
Last State
Last Action
Process Information
Timestamp
Screenshot
Logs
```

---

33. Hang Detection

A hang occurs when the application stops responding within an allowed timeout.

Possible signals:

```
Window Not Responding
Action Timeout
Observation Timeout
UI Automation Timeout
```

The initial deterministic rule can be:

```
Action or observation exceeds configured timeout
        +
Application process still exists
        =
Potential Hang
```

The framework should distinguish:

```
Slow Operation
```

from:

```
Application Hang
```

using configurable thresholds.

---

34. Disappearing Window Detection

The process may remain alive while the main application window disappears.

Example:

```
Process Alive = True
Main Window Exists = False
```

This should be recorded separately from a process crash.

Possible causes:

- Application closed its main window

- Application navigated to an unexpected window
- Window crashed while process remained alive
- Automation lost the application target

---

35. Action Execution Failure

An execution failure occurs when the automation driver cannot perform the requested action.

Examples:

```
Control Not Found
Control Disabled
Click Failed
Input Rejected
Automation Exception
```

Execution failures are not automatically application bugs.

They may indicate:

```
Framework Bug
Stale State
Changed Application State
Automation Limitation
Actual Application Failure
```

The failure must be recorded for later classification.

---

36. Replay Mismatch Detection

During replay:

```
Expected State != Actual State
```

The system should create:

```
REPLAY_MISMATCH
```

Evidence:

```
Expected State Hash
Actual State Hash
Replay Step
Action
Expected Screenshot
Actual Screenshot
```

This failure is particularly important for detecting nondeterministic behavior.

---

37. Unexpected Error Dialog Detection

The framework should maintain known patterns for error windows.

Examples:

```
Error
Exception
Fatal Error
Application Error
Unhandled Exception
```

Initial detection should use deterministic window metadata.

Later, semantic analysis can determine whether the dialog represents:

```
Expected Validation Message
```

or:

```
Unexpected Application Failure
```

---

38. Validation Architecture

Failure detection and validation are related but separate.

Failure detection asks:

```
Did something obviously break?
```

Validation asks:

```
Was the observed behavior correct?
```

Architecture:

```
Observed Transition
        |
        V
Deterministic Validators
        |
        V
Semantic Validators
        |
        V
Validation Results
```

---

39. Deterministic Validators First

Initial validators should be deterministic.

Examples:

```
Expected Process Alive
Expected Window Present
Expected Display Value
Expected State Change
Expected Control Enabled
Expected Error Message
```

Validator interface:

```python
class Validator(ABC):

    @abstractmethod
    def validate(
        self,
```

```
        context: ValidationContext
    ) -> ValidationResult:
        pass
```

---

40. Validation Result

```
@dataclass
class ValidationResult:
    validator_id: str
    passed: bool
    expected: object
    actual: object
    message: str
    evidence: dict
```

Possible outcomes:

```
PASS
FAIL
INCONCLUSIVE
NOT_APPLICABLE
```

---

41. Semantic Validators

Semantic validators should be added later.

Examples:

```
Does this error message make sense?

Is the observed behavior consistent with the requirement?

Is the UI response confusing?

Is this result logically incorrect?
```

These validators may use:

```
Local LLM
Domain Rules
Historical Knowledge
Requirement Context
```

Semantic validation must remain behind the same validator interface.

---

42. Requirement Model

The testing agent eventually needs to understand more than raw application states.

It needs to represent:

```
Goals
Workflows
Preconditions
Actions
Expected Outcomes
Coverage
```

A requirement is a testable statement about expected system behavior.

---

43. Requirement Object

```python
@dataclass
class Requirement:
    requirement_id: str
    title: str
    description: str
    preconditions: list[Precondition]
    goals: list[Goal]
    expected_outcomes: list[ExpectedOutcome]
    workflows: list[Workflow]
```

---

44. Goal Model

A goal describes what the agent should achieve.

Example:

```
Perform addition of two positive numbers.
```

Model:

```python
@dataclass
class Goal:
    goal_id: str
    description: str
    completion_conditions: list[Condition]
```

---

45. Workflow Model

A workflow describes a meaningful behavioral path.

Example:

```
Start Calculator
        ↓
Enter First Number
        ↓
Select Addition
        ↓
Enter Second Number
        ↓
Calculate Result
```

Model:

```python
@dataclass
class Workflow:
    workflow_id: str
    name: str
    steps: list[WorkflowStep]
```

---

46. Preconditions

Preconditions define required starting conditions.

Examples:

```
Calculator is running.

Calculator is in Standard mode.

Display contains 0.
```

The agent should verify preconditions before executing requirement-driven testing.

---

47. Expected Outcomes

Expected outcomes define correct behavior.

Example:

```
Given:
2 + 3

Expected:
5
```

Model:

```
@dataclass
class ExpectedOutcome:
    description: str
    validator_id: str
    expected_value: object
```

---

48. Requirement Coverage

Coverage should eventually include:

```
State Coverage
Action Coverage
Transition Coverage
Requirement Coverage
Workflow Coverage
Goal Coverage
```

Requirement coverage asks:

```
Which requirements were tested?

Which requirements passed?

Which requirements failed?

Which requirements remain unexplored?
```

---

49. AI Integration Architecture

AI should be integrated behind stable interfaces.

It should not be embedded throughout the framework.

Incorrect architecture:

```
Driver calls LLM
Graph calls LLM
Executor calls LLM
Validator calls LLM
Database calls LLM
```

Recommended architecture:

```
Core Framework
        |
        V
Strategy Interfaces
        |
        V
AI Implementations
```

```
        |
        V
Local Model Provider
```

---

50. Local Model Provider

The initial model provider will use Ollama.

Initial planned model:

```
qwen2.5-coder:7b
```

The model should be accessed through an abstraction.

```python
class LanguageModelProvider(ABC):

    @abstractmethod
    def generate(
        self,
        request: ModelRequest
    ) -> ModelResponse:
        pass
```

Implementation:

```
OllamaLanguageModelProvider
```

This allows future support for other local or remote models without changing the core testing framework.

---

51. AI Strategy Interfaces

AI should initially be used for five strategic tasks:

```
Target Ranking
Action Selection
Workflow Planning
```

```
Requirement Interpretation
Failure Analysis
```

Each capability should have its own interface.

---

52. AI-Guided Target Ranking

The deterministic explorer may have many states with unexplored actions.

AI can rank which state should be explored next.

Input:

```
Current Graph
Coverage
Known Failures
Requirements
Historical Sessions
```

Output:

```
Ranked Exploration Targets
```

The AI does not restore or execute the target.

It only ranks candidates.

---

53. AI-Guided Action Selection

The AI selector implements the same interface as the deterministic selector.

```
ActionSelector
    |
    +---- DeterministicActionSelector
    |
    +---- RandomActionSelector
    |
    +---- HeuristicActionSelector
```

```
    |
    +---- AIActionSelector
```

The coordinator remains unchanged.

This is the key architectural benefit of strategy interfaces.

---

54. AI-Guided Workflow Planning

AI can generate higher-level plans.

Example:

```
Goal:
Test calculator overflow behavior.

Plan:

1. Enter a very large number.
2. Select multiplication.
3. Enter another large number.
4. Calculate result.
5. Inspect display and application stability.
```

The deterministic framework converts the plan into executable actions.

AI plans.

The framework executes.

---

55. Requirement Interpretation

AI can transform natural-language requirements into structured models.

Input:

```
The calculator should display an error when division by zero is attempted.
```

Output:

```
Requirement

Goal:
Attempt division by zero.

Precondition:
Calculator is ready.

Workflow:
Enter non-zero number.
Press divide.
Enter zero.
Press equals.

Expected Outcome:
Error state is displayed.
Application remains responsive.
```

The structured result should then be stored persistently.

---

56. AI-Guided Failure Analysis

After deterministic failure detection, AI can analyze evidence.

Input:

```
Source State
Action
Target State
Failure Type
Screenshot
Historical Failures
Requirement Context
```

Output:

```
Possible Root Cause
Severity Estimate
Failure Explanation
Suggested Reproduction Path
Potential Duplicate Bug
```

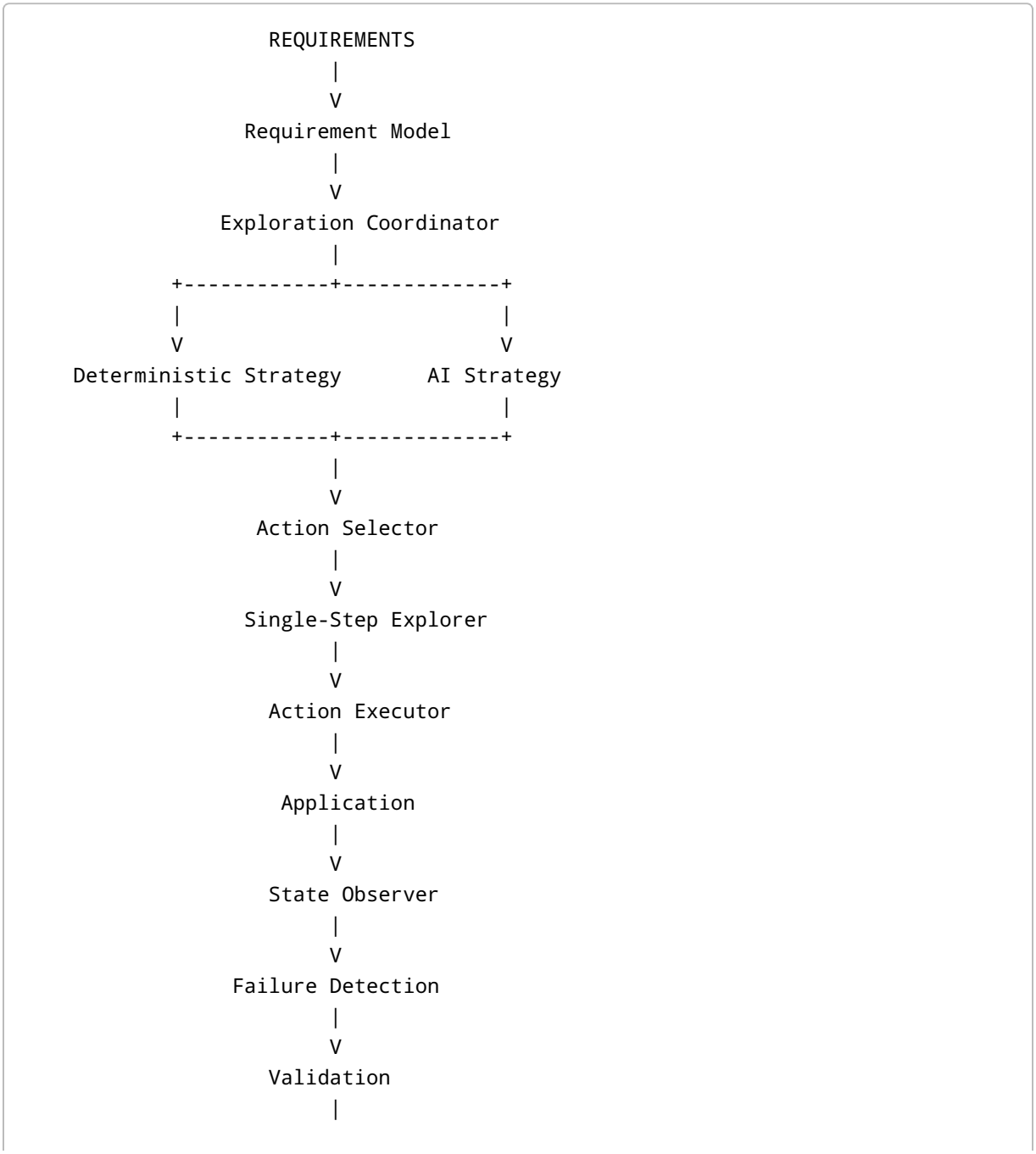
AI should not decide whether a crash occurred.

The deterministic detector already knows that.

AI explains and classifies the crash.

---

57. Complete Final Architecture



```
                V
          State Graph
                |
                V
           Persistence
                |
                V
       Historical Memory
                |
                V
          AI Reasoning
```

---

58. Recommended Implementation Milestones

Milestone 1: Deterministic Action Selector

Implement:

```
Action identity
Action candidate
Selection context
Selector interface
Deterministic selector
State-action coverage
```

Success condition:

```
The same context always produces the same selected action.
```

Milestone 2: Single Exploration Step

Implement:

```
Observe
Select
Execute
Observe
```

```
Record
Return Result
```

Success condition:

```
One method call performs exactly one complete exploration transaction.
```

Milestone 3: Closed Coordinator Loop

Implement:

```
Repeated exploration steps
Target selection
State restoration
Replay
Termination conditions
```

Success condition:

```
The agent explores automatically until a deterministic stop condition is
reached.
```

Milestone 4: Persistence

Implement:

```
Session storage
State storage
Transition storage
Memory storage
Screenshot storage
Resume
```

Success condition:

```
An interrupted exploration session can continue without losing discovered
knowledge.
```

Milestone 5: Failure Detection

Implement:

```
Crash detection
Hang detection
Window disappearance
Execution failures
Replay mismatches
Error dialog detection
```

Success condition:

```
Known infrastructure-level failures are detected without AI.
```

Milestone 6: Validation

Implement:

```
Validator interface
Validation context
Deterministic validators
Validation results
Semantic validator extension point
```

Success condition:

```
Expected and actual behavior can be compared through pluggable validators.
```

Milestone 7: Requirement Model

Implement:

```
Requirements
Goals
Workflows
Preconditions
Expected outcomes
Requirement coverage
```

Success condition:

```
The system can connect exploration results to explicit testing objectives.
```

Milestone 8: Local AI Integration

Implement:

```
LLM provider interface
Ollama provider
Structured prompts
Structured responses
Decision logging
```

Initial model:

```
qwen2.5-coder:7b
```

Success condition:

```
AI capabilities can be enabled or disabled without changing the deterministic
framework.
```

Milestone 9: AI-Guided Testing

Implement:

```
Target ranking
Action selection
Workflow planning
```

```
Requirement interpretation
Failure analysis
```

Success condition:

```
AI improves exploration efficiency while deterministic execution, persistence,
and validation remain authoritative.
```

---

59. Final Architectural Principle

The Free Form Testing Agent should evolve in layers.

```
Layer 1:
Deterministic Application Interaction

Layer 2:
State and Transition Modeling

Layer 3:
Deterministic Exploration

Layer 4:
Persistent Historical Knowledge

Layer 5:
Failure Detection and Validation

Layer 6:
Requirements and Testing Intent

Layer 7:
AI-Guided Strategy
```

The project should never depend on AI for basic correctness.

If the local model is unavailable, the system should still be able to:

```
Launch
Observe
Explore
Execute
```

```
Persist
Resume
Detect Failures
Validate
Measure Coverage
Generate Evidence
```

AI is added to improve:

```
Prioritization
Planning
Interpretation
Analysis
Efficiency
```

This architecture creates a stable path from the current deterministic state-graph explorer to a genuinely intelligent autonomous testing agent.